Project Report

ECHO DETECTIVE

A C++ Echo Detection Simulator Using Normalized Cross-Correlation

---

Submitted by: Het Patel
Roll No.: 25EC412
Branch: Electronics and Communication
Institute: Birla Vishvakarma Mahavidyalaya
Guided by: Mayur Sevak

Date: October 24, 2026

Academic Year 2026–27

Abstract

Echoes are delayed and attenuated copies of a signal that arrive after reflection from surfaces. Detecting the delay of an echo is a fundamental task in acoustics, sonar, radar, telephony (echo cancellation) and room-acoustic analysis. This project, *Echo Detective*, is a self-contained C++ program that (i) synthesizes a known test signal, (ii) adds an echo of known delay and gain, (iii) corrupts the result with reproducible Gaussian noise, and (iv) estimates the echo delay by searching for the lag that maximizes the normalized correlation between the original and the observed signal. For the chosen parameters (8 kHz sampling, 0.25 s true delay, echo gain 0.45, noise standard deviation 0.08) the program recovers the delay exactly (0.0 ms error) with a correlation score of 0.9932. The report explains the theory, design, flowcharts, implementation, results and limitations of the method.

Contents

1 Introduction **2**

2 Objectives **2**

3 Theory **2**
3.1 Signal model 2
3.2 Test signal 2
3.3 Normalized correlation 3
3.4 Search range and cost 3

4 System Design **3**

5 Flowcharts **5**
5.1 Overall program flow 5
5.2 Flowchart of `detectEcho()` 6

6 Implementation Notes **7**

7 Results **7**
7.1 Correlation score at selected lags 7

8 Discussion and Limitations **8**

9 Future Scope **8**

10 Conclusion **9**

A Complete Source Code **9**

1 Introduction

Whenever sound travels in an enclosed space, part of it reaches the listener directly and part arrives later after reflections. The delayed components are heard as echoes. In engineering systems, estimating the delay of such a reflection is important for echo cancellation in telephony, range estimation in sonar and radar, and acoustic analysis of rooms.

A simple and widely used approach is *correlation-based delay estimation*: slide a delayed copy of the known source signal across the observed signal and measure how well they match at every lag. The lag giving the strongest match is the echo delay. *Echo Detective* implements this idea in portable standard C++ with no external libraries, so the entire pipeline can be executed and verified on any computer.

2 Objectives

- To generate a repeatable multi-tone test signal in software.
- To simulate an echo with a user-defined delay and gain.
- To simulate additive white Gaussian noise with a fixed random seed, so results are reproducible.
- To estimate the echo delay using normalized cross-correlation.
- To quantify accuracy through the delay error in milliseconds.

3 Theory

3.1 Signal model

Let $x[n]$ be the original signal sampled at f_s. The observed signal containing a single echo and noise is

$$y[n] = x[n] + \alpha\, x[n - D] + w[n], \tag{1}$$

where D is the echo delay in samples, α is the echo gain ($0 < \alpha < 1$) and $w[n] \sim \mathcal{N}(0, \sigma^2)$ is white Gaussian noise. The delay in seconds is $\tau = D/f_s$.

3.2 Test signal

The original signal is a sum of three sinusoids (harmonically related tones):

$$x[n] = 0.60 \sin(2\pi\, 440\, t_n) + 0.30 \sin(2\pi\, 660\, t_n) + 0.10 \sin(2\pi\, 880\, t_n), \qquad t_n = n/f_s. \tag{2}$$

3.3 Normalized correlation

For a candidate lag d the program computes

$$\rho(d) = \frac{\sum_{i=d}^{N-1} x[i-d]\, y[i]}{\sqrt{\sum_{i=d}^{N-1} x^2[i-d] \sum_{i=d}^{N-1} y^2[i]}}, \tag{3}$$

which is the cosine of the angle between the delayed original and the observed signal, so $-1 \le \rho \le 1$. A value close to 1 means the two waveforms are well aligned at that lag. The estimated delay is

$$\hat{D} = \arg \max_{d_{\min} \le d \le d_{\max}} \rho(d), \tag{4}$$

where $d_{\min}$ excludes lag 0 (the direct sound, which would always match best) and $d_{\max}$ limits the search range.

3.4 Search range and cost

The search runs from $d_{\min} = \lfloor 0.05 f_s \rfloor = 400$ to $d_{\max} = \lfloor 0.60 f_s \rfloor = 4800$ samples, i.e. 4401 candidate lags. Each lag requires $N - d$ multiply–accumulate iterations, giving a total of about 5.9×10^7 iterations for $N = 16000$. The complexity is $O(N \cdot L)$ for L lags; this is quick for the demo but could be reduced to $O(N \log N)$ using FFT-based correlation.

4 System Design

The program is organized into five functions plus `main()`.

Table 1: Functions of the program

Function	Purpose
`generateSignal`	Builds the 440/660/880 Hz three-tone test signal of given length and sample rate.
`addEcho`	Returns a copy of the signal with a delayed, scaled copy added: `result[i] += gain * original[i-delay]`.
`addNoise`	Adds Gaussian noise from `mt19937` seeded with a fixed value (12345) for reproducibility.
`detectEcho`	Scans all lags in the allowed range, computes $\rho(d)$ and returns the best lag and score (struct `EchoResult`).
`main`	Sets parameters, calls the above functions, prints results and decides success if error ≤ 10 ms.

Table 2: Simulation parameters used in `main()`

Parameter	**Value**	**Meaning**
Sample rate f_s	8000 Hz	Samples per second
Duration	2.0 s	Signal length ($N = 16000$ samples)
True delay	0.25 s	$D = 2000$ samples
Echo gain α	0.45	Echo amplitude relative to original
Noise std. dev. σ	0.08	Gaussian noise amplitude
Noise seed	12345	Makes runs repeatable
Search range	0.05–0.60 s	400 to 4800 samples
Success threshold	10 ms	Maximum accepted delay error

5 Flowcharts

5.1 Overall program flow

Figure 1 shows the complete execution of `main()`.

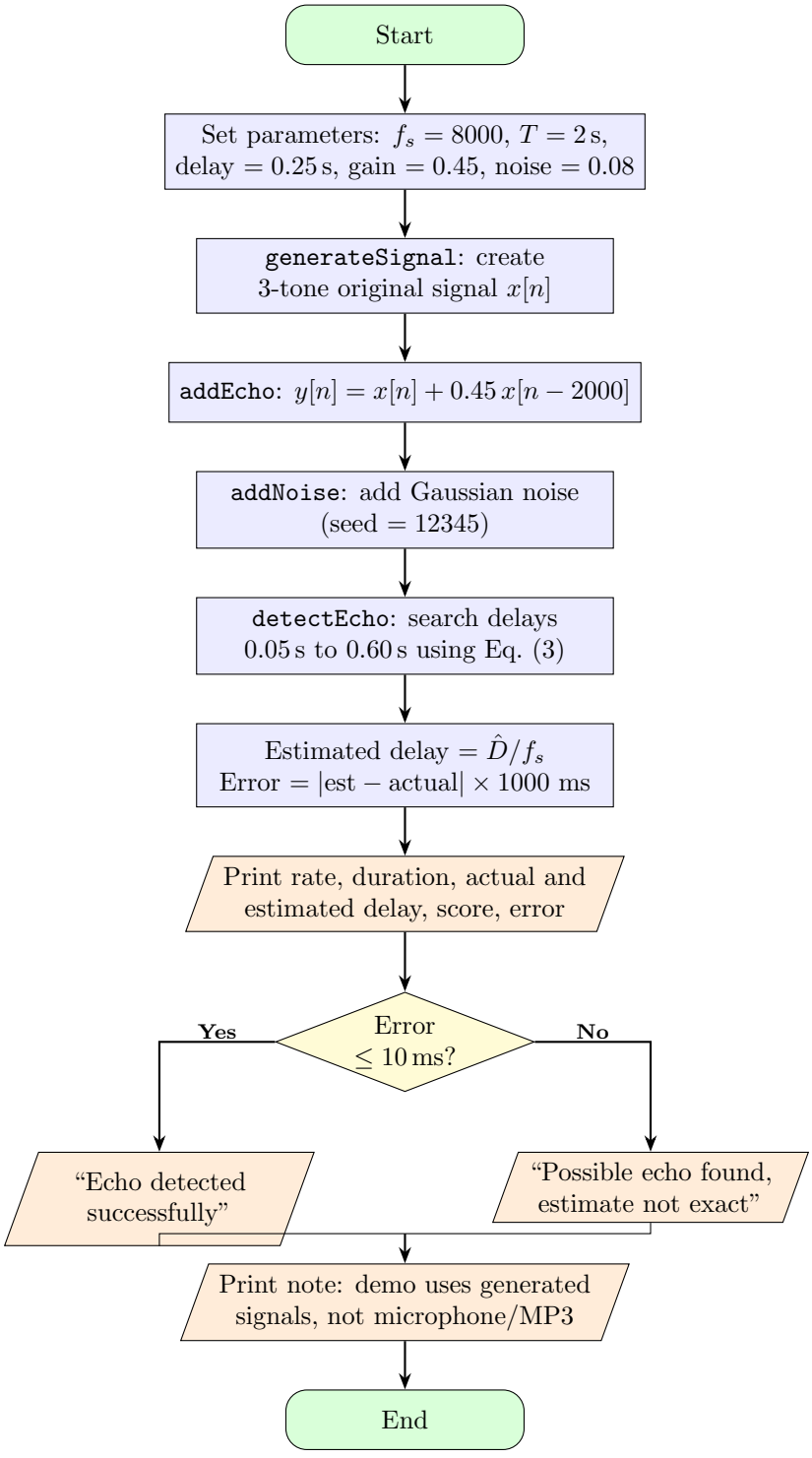


Figure 1: Flowchart of the main program

5.2 Flowchart of `detectEcho()`

The core of the project is the correlation search. Figure 2 shows how `detectEcho` loops over every candidate lag.

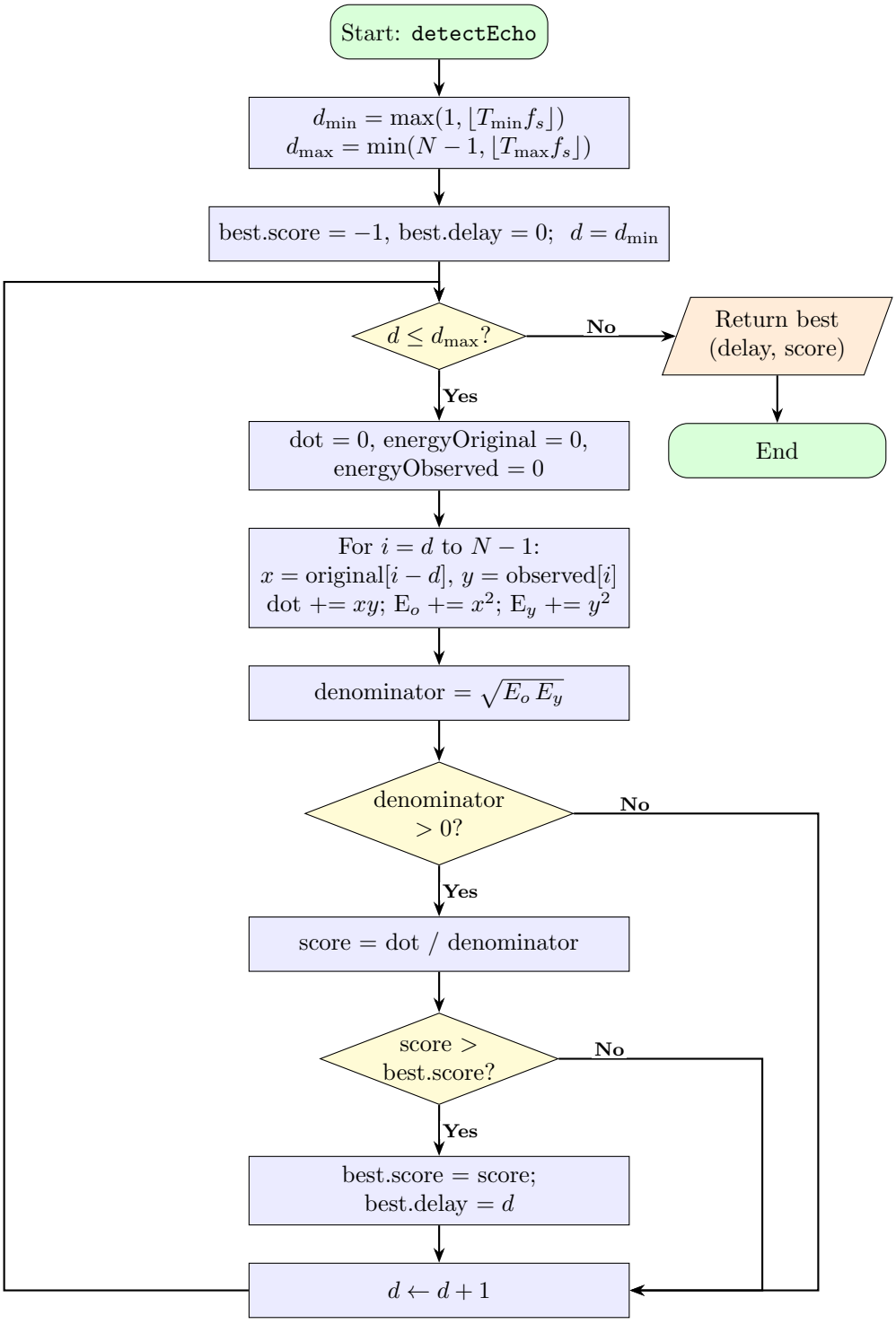


Figure 2: Flowchart of the `detectEcho` function

6 Implementation Notes

- **Language and libraries:** standard C++ only (`<vector>`, `<cmath>`, `<random>`, `<iomanip>`, `<iostream>`, `<algorithm>`).
- **Reproducibility:** the Mersenne Twister engine `mt19937` is seeded with a fixed value, so every run gives identical output.
- **Lag 0 is skipped:** the direct (non-delayed) signal always matches perfectly, so the search starts at 0.05 s.
- **Normalization:** dividing by the energies makes the score independent of signal loudness and limits it to $[-1, 1]$.
- **Result decision:** an estimate within 10 ms of the true delay is reported as a successful detection.

7 Results

Compiling with `g++ -O2 -std=c++17` and running the program produced:

Table 3: Program output

Quantity	**Value**
Sample rate	8000 samples/s
Signal duration	2.0000 s
Actual echo delay	0.2500 s
Estimated echo delay	0.2500 s
Correlation score	0.9932
Delay estimation error	0.0000 ms
Verdict	Echo detected successfully

The estimated delay equals the true delay of 2000 samples, so the error is zero, well below the 10 ms threshold. The high score of 0.9932 confirms that the delayed original matches the observed signal very closely; the small deficit from 1 is caused by the added noise.

7.1 Correlation score at selected lags

To understand the behaviour of the detector, the score of Eq. (3) was also evaluated at a few other lags using the same signal, echo and noise seed.

Table 4: Normalized correlation $\rho(d)$ at selected lags

Lag d (samples)	**Delay (s)**	$\rho(d)$
400	0.0500	0.9883
800	0.1000	0.9894
1200	0.1500	0.9906
1600	0.2000	0.9918
2000	**0.2500**	**0.9932**
2400	0.3000	0.9932
4800	0.6000	0.9932
401	0.0501	0.9126
1000	0.1250	0.6028

8 Discussion and Limitations

1. **Periodic test signal.** The three tones (440, 660 and 880 Hz) are all integer multiples of 20 Hz, so the signal repeats exactly every 0.05 s (400 samples). As Table 4 shows, every lag that is a multiple of 400 samples gives a nearly identical score (0.988–0.993), and the scores for 2000 and beyond agree to four decimal places. The program selects 0.25 s because the comparison `score > best.score` is strict and the score is highest there; the margin over neighbouring peaks is very small. The result is correct for this demo, but the method is not robust against such ambiguity.

2. **Single echo only.** The signal model has one echo; real rooms produce many reflections.

3. **Known original signal.** The detector needs the clean source signal as a reference, which is not always available in practice.

4. **Integer-sample resolution.** The delay resolution is $1/f_s = 0.125$ ms; sub-sample delays would need interpolation.

5. **Computation time.** Direct evaluation costs about 5.9×10^7 iterations here; FFT-based correlation would be faster for long signals.

6. **Synthetic data only.** The program does not read microphone or MP3 audio.

9 Future Scope

- Use a non-periodic probe signal (a chirp, an MLS sequence or white noise) so that the correlation has a single sharp peak.
- Compute the correlation by FFT to cut the run time.
- Detect multiple echoes by picking several peaks and estimating each gain.
- Read real recordings from WAV files and apply autocorrelation when no reference signal is available.
- Implement an adaptive filter (LMS/NLMS) to cancel the detected echo.

10 Conclusion

Echo Detective shows that an echo delay can be recovered from a noisy observation by maximizing normalized correlation over a range of lags. With a true delay of 0.25 s, an echo gain of 0.45 and noise of 0.08, the program estimated the delay exactly and achieved a correlation score of 0.9932. The project also demonstrated why the choice of probe signal matters: a periodic signal produces repeated correlation peaks, so a non-periodic signal is recommended for more reliable detection.

References

1. A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.

2. J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms and Applications*, 4th ed., Pearson, 2007.

3. S. Haykin, *Adaptive Filter Theory*, 5th ed., Pearson, 2013.

4. B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley, 2013.

A Complete Source Code

```

#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <random>
#include <string>
#include <vector>

using namespace std;

constexpr double PI = 3.14159265358979323846;

// A simple combination of sine waves makes a repeatable test signal.
vector<double> generateSignal(int sampleCount, int sampleRate) {
    vector<double> signal(sampleCount, 0.0);

    for (int i = 0; i < sampleCount; ++i) {
        const double t = static_cast<double>(i) / sampleRate;
        signal[i] =
            0.60 * sin(2.0 * PI * 440.0 * t) +
            0.30 * sin(2.0 * PI * 660.0 * t) +
            0.10 * sin(2.0 * PI * 880.0 * t);
    }
    return signal;
}

// Add a delayed, quieter copy of the original signal.
// delaySamples determines the echo delay; echoGain controls its loudness.
vector<double> addEcho(const vector<double>& original,
                       int delaySamples,
                       double echoGain) {
    vector<double> result = original;

    for (int i = delaySamples; i < static_cast<int>(original.size()); ++i) {
        result[i] += echoGain * original[i - delaySamples];
    }
    return result;
```

```cpp
}

// Add reproducible random noise so results are consistent across runs.
void addNoise(vector<double>& signal, double noiseAmplitude, unsigned seed) {
    mt19937 generator(seed);
    normal_distribution<double> noise(0.0, noiseAmplitude);

    for (double& sample : signal) {
        sample += noise(generator);
    }
}

// For each possible delay, compare y[t] with x[t-delay].
// The normalized correlation score is near 1 when the delayed patterns align.
// We skip delay 0 because we are searching for an echo, not the direct signal.
struct EchoResult {
    int delaySamples = 0;
    double score = -1.0;
};

EchoResult detectEcho(const vector<double>& original,
                      const vector<double>& observed,
                      int sampleRate,
                      double minDelaySeconds,
                      double maxDelaySeconds) {
    const int minDelay = max(1, static_cast<int>(minDelaySeconds * sampleRate));
    const int maxDelay = min(
        static_cast<int>(original.size()) - 1,
        static_cast<int>(maxDelaySeconds * sampleRate)
    );

    EchoResult best;

    for (int delay = minDelay; delay <= maxDelay; ++delay) {
        double dot = 0.0;
        double energyOriginal = 0.0;
        double energyObserved = 0.0;

        for (int i = delay; i < static_cast<int>(original.size()); ++i) {
            const double x = original[i - delay];
            const double y = observed[i];

            dot += x * y;
            energyOriginal += x * x;
            energyObserved += y * y;
        }

        const double denominator =
            sqrt(energyOriginal * energyObserved);

        if (denominator > 0.0) {
            const double score = dot / denominator;
            if (score > best.score) {
                best.score = score;
                best.delaySamples = delay;
            }
        }
    }

    return best;
}

int main() {
    const int sampleRate = 8000;        // samples per second
    const double durationSeconds = 2.0;
    const int sampleCount =
        static_cast<int>(sampleRate * durationSeconds);

    // Known echo settings for this demonstration.
    const double actualDelaySeconds = 0.25;
    const int actualDelaySamples =
        static_cast<int>(actualDelaySeconds * sampleRate);
    const double echoGain = 0.45;
```

```
    const double noiseAmplitude = 0.08;

    cout << "====================================\n";
    cout << "          ECHO DETECTIVE\n";
    cout << "   C++ Echo Detection Simulator\n";
    cout << "====================================\n\n";

    cout << "Generating test signal...\n";
    vector<double> original = generateSignal(sampleCount, sampleRate);

    cout << "Adding an echo and random noise...\n";
    vector<double> observed = addEcho(original, actualDelaySamples, echoGain);
    addNoise(observed, noiseAmplitude, 12345);

    cout << "Searching for echo delay...\n\n";
    EchoResult result = detectEcho(
        original, observed, sampleRate, 0.05, 0.60
    );

    cout << fixed << setprecision(4);
    cout << "Sample rate          : " << sampleRate << " samples/second\n";
    cout << "Signal duration      : " << durationSeconds << " seconds\n";
    cout << "Actual echo delay    : " << actualDelaySeconds << " seconds\n";
    cout << "Estimated echo delay : "
         << static_cast<double>(result.delaySamples) / sampleRate
         << " seconds\n";
    cout << "Correlation score    : " << result.score << "\n\n";

    const double estimatedDelay =
        static_cast<double>(result.delaySamples) / sampleRate;
    const double errorMs =
        abs(estimatedDelay - actualDelaySeconds) * 1000.0;

    cout << "Delay estimation error: " << errorMs << " ms\n";

    if (errorMs <= 10.0) {
        cout << "Result: Echo detected successfully.\n";
    } else {
        cout << "Result: A possible echo was found, but the estimate is not exact.\n";
    }

    cout << "\nNote: This demo uses generated signals, not microphone or MP3 audio.\n";
    return 0;
}
```